

CLOUD COMPUTING

UNIT III & CLOUD DOCKER

TOPIC: DOCKER ARCHITECTURE

1. DOCKER ARCHITECTURE:

- Docker makes use of a client-server architecture.
- The Docker client talks with the docker daemon which helps in building, running, and distributing the docker containers.
- The Docker client runs with the daemon on the same system or we can connect the Docker client with the Docker daemon remotely.
- With the help of REST API over a UNIX socket or a network, the docker client and daemon interact with each other.

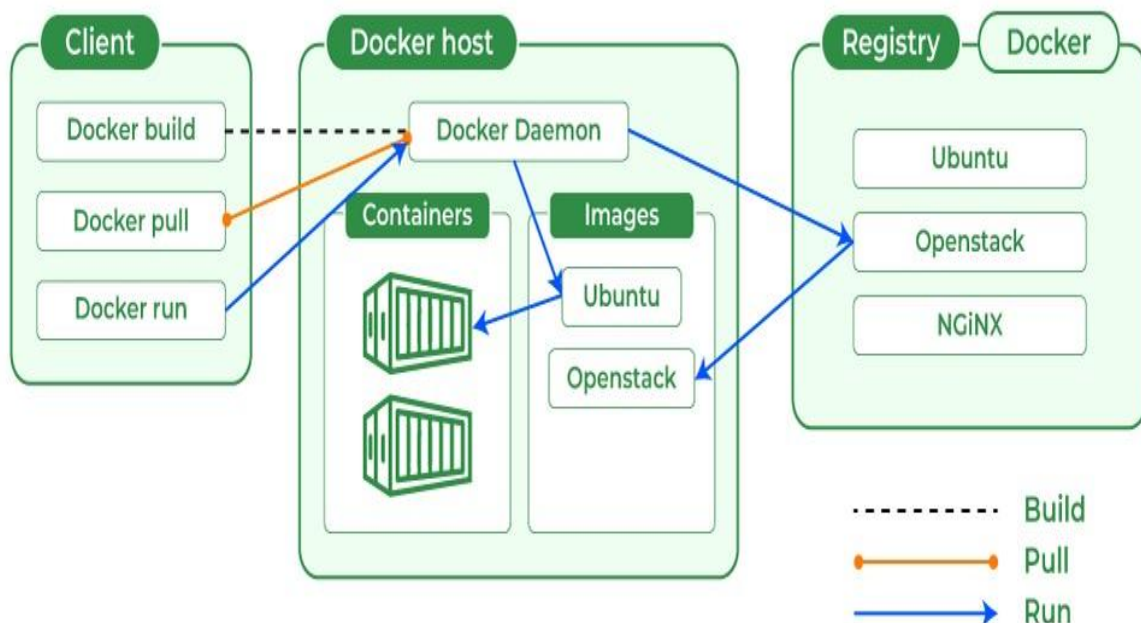


Fig 1.1 Docker Architecture

How Docker Works

The Docker platform revolves around the **Docker Engine**, which manages the lifecycle of containers. Its workflow involves:

1. **Docker Daemon:**
 - A background process that builds, runs, and manages Docker objects such as containers, images, networks, and volumes.
 - Processes API requests and performs container-related tasks.
2. **Docker REST API:**
 - Enables communication between applications and the Docker Daemon.
 - Facilitates interactions through HTTP clients.
3. **Docker CLI:**
 - A command-line tool that allows users to interact with the Docker Daemon.
 - Simplifies the management of Docker containers and related objects.

The Docker client communicates with the Docker Daemon using the REST API, sending commands that the Daemon executes. This interaction can occur on the same system or remotely over a network.

1.1 What is Docker Daemon?

- Docker daemon manages all the services by communicating with other daemons.
- It manages docker objects such as images, containers, networks, and volumes with the help of the API requests of Docker.

1.2 Docker Client

- With the help of the docker client, the docker users can interact with the docker. The docker command uses the Docker API.
- The Docker client can communicate with multiple daemons.
- When a docker client runs any docker command on the docker terminal then the terminal sends instructions to the daemon.
- The Docker daemon gets those instructions from the docker client withinside the shape of the command and REST API's request.

- The main objective of the docker client is to provide a way to direct the pull of images from the docker registry and run them on the docker host.
- The common commands which are used by clients are docker build, docker pull, and docker run.

1.3 Docker Host

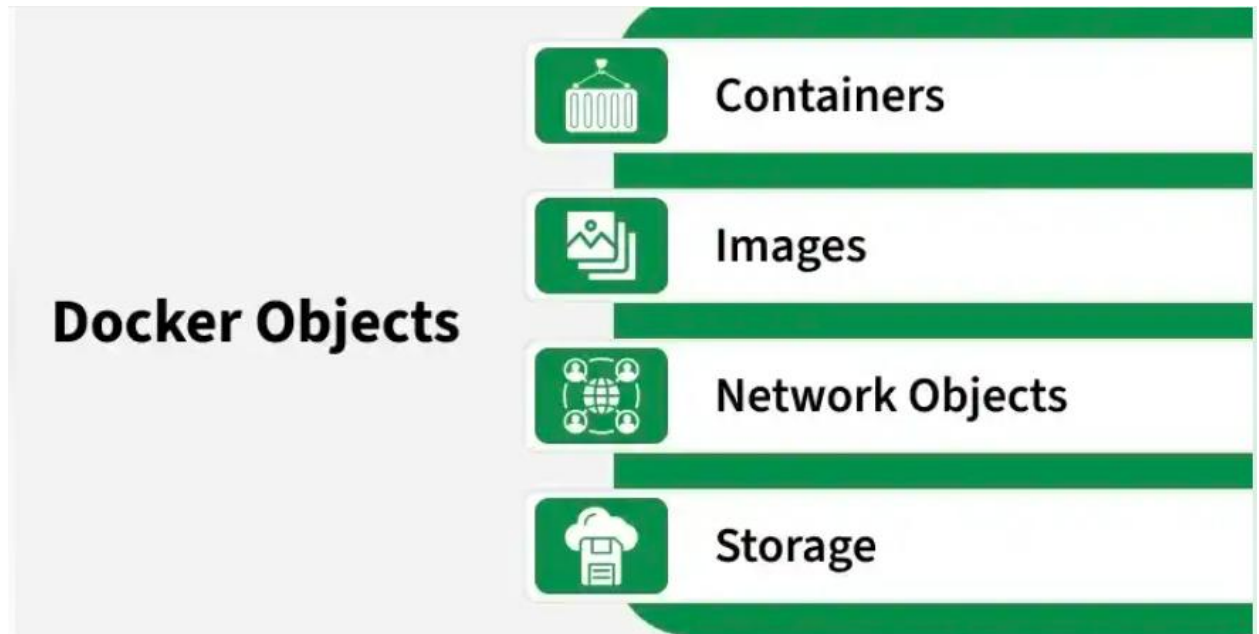
- A Docker host is a type of machine that is responsible for running more than one container.
- It comprises the Docker daemon, Images, Containers, Networks, and Storage.

1.4 Docker Registry

- All the docker images are stored in the docker registry.
- There is a public registry which is known as a docker hub that can be used by anyone. We can run our private registry also.
- With the help of docker run or docker pull commands, we can pull the required images from our configured registry.
- Images are pushed into configured registry with the help of the docker push command.

1.5 Docker Objects

- Whenever we are using a docker, we are creating and use images, containers, volumes, networks, and other objects.



Docker objects:-

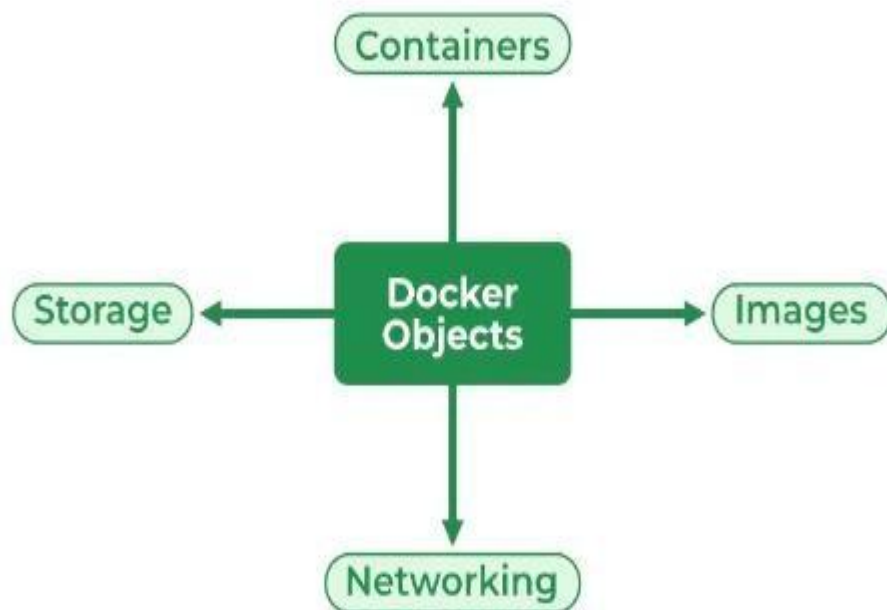
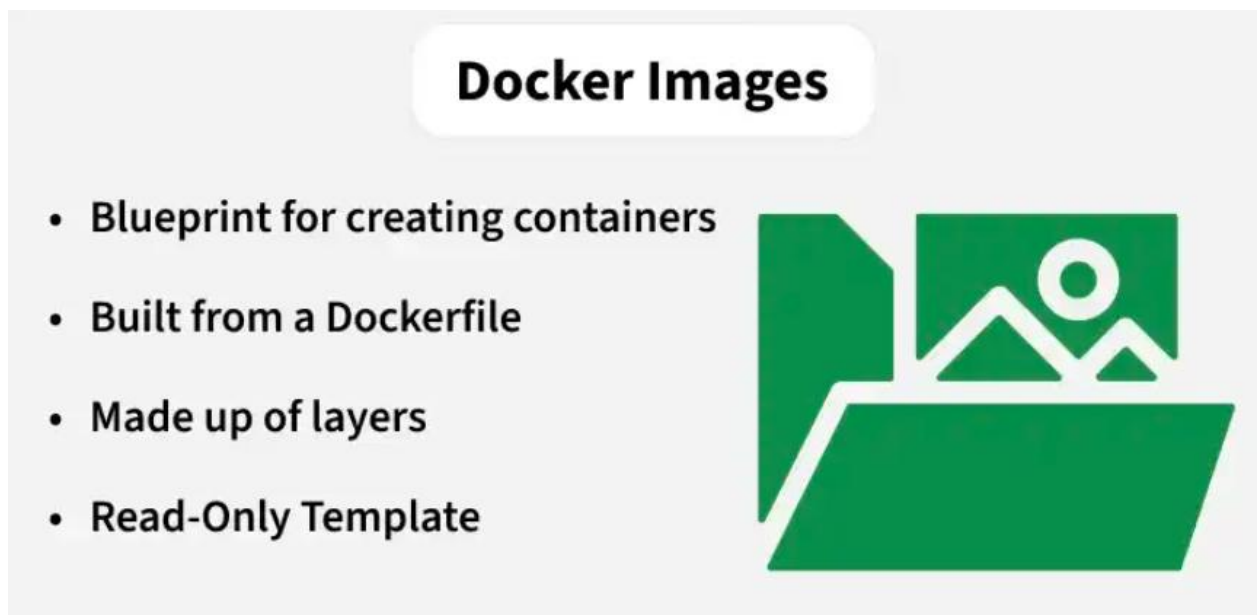


Fig 1.2 Docker Objects

1.6 Docker Images

- An image contains instructions for creating a docker container.
- It is just a read-only template.
- It is used to store and ship applications.
- Images are an important part of the docker experience as they enable collaboration between developers in any way which is not possible earlier.



1.7 Docker Containers

- Containers are created from docker images as they are ready applications.
- With the help of Docker API or CLI, we can start, stop, delete, or move a container.
- A container can access only those resources which are defined in the image unless additional access is defined during the building of an image in the container.

1.8 Docker Storage

- We can store data within the writable layer of the container but it requires a storage driver.
- Storage driver controls and manages the images and containers on our docker host.

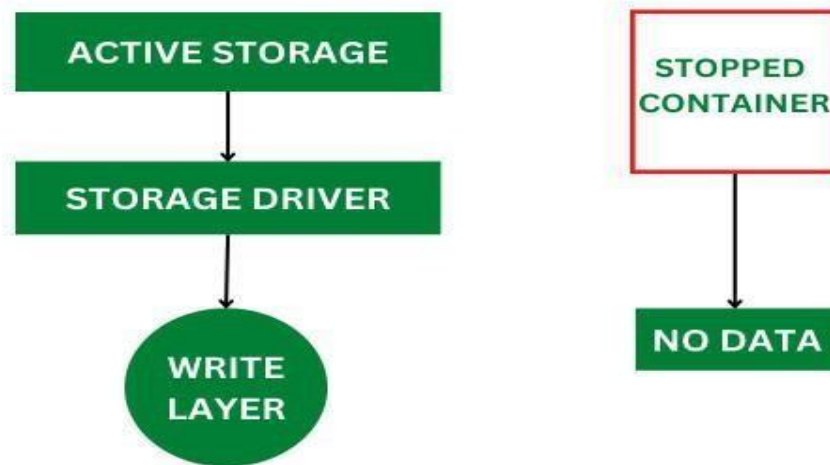


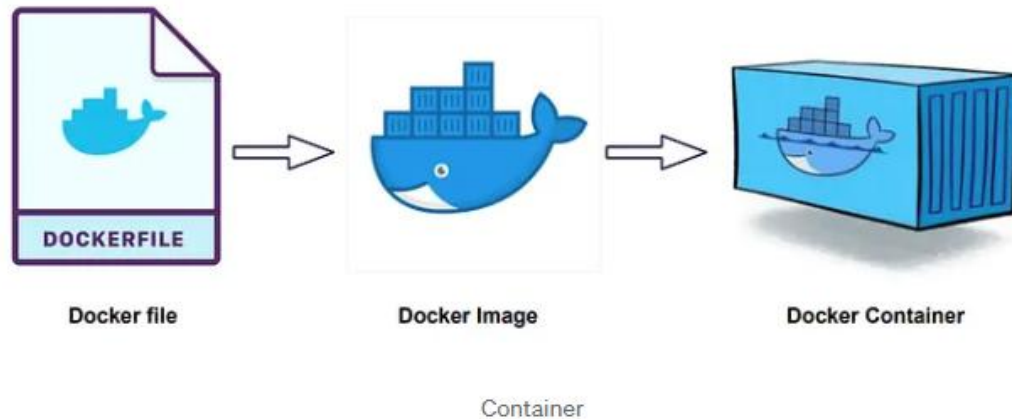
Fig 1.3 Docker Storage

Types of Docker Storage

1. **Data Volumes:** Data Volumes can be mounted directly into the filesystem of the container and are essentially directories or files on the Docker Host filesystem.
2. **Volume Container:** In order to maintain the state of the containers (data) produced by the running container, Docker volumes file systems are mounted on Docker containers. independent container life cycle, the volumes are stored on the host. This makes it simple for users to exchange file systems among containers and backup data.
3. **Directory Mounts:** A host directory that is mounted as a volume in your container might be specified.
4. **Storage Plugins:** Docker volume plugins enable us to integrate the Docker containers with external volumes like Amazon EBS by this we can maintain the state of the container.

Docker Networking

Docker networking provides complete isolation for docker containers. It means a user can link a docker container to many networks. It requires very less OS instances to run the workload.



Docker provides networking capabilities for containers to communicate with each other and with external networks. It uses software-defined networks (SDN) to create virtual networks, enabling connectivity and isolation. Users can create custom networks, connect containers to networks, and define network policies using Docker commands or Docker Compose.

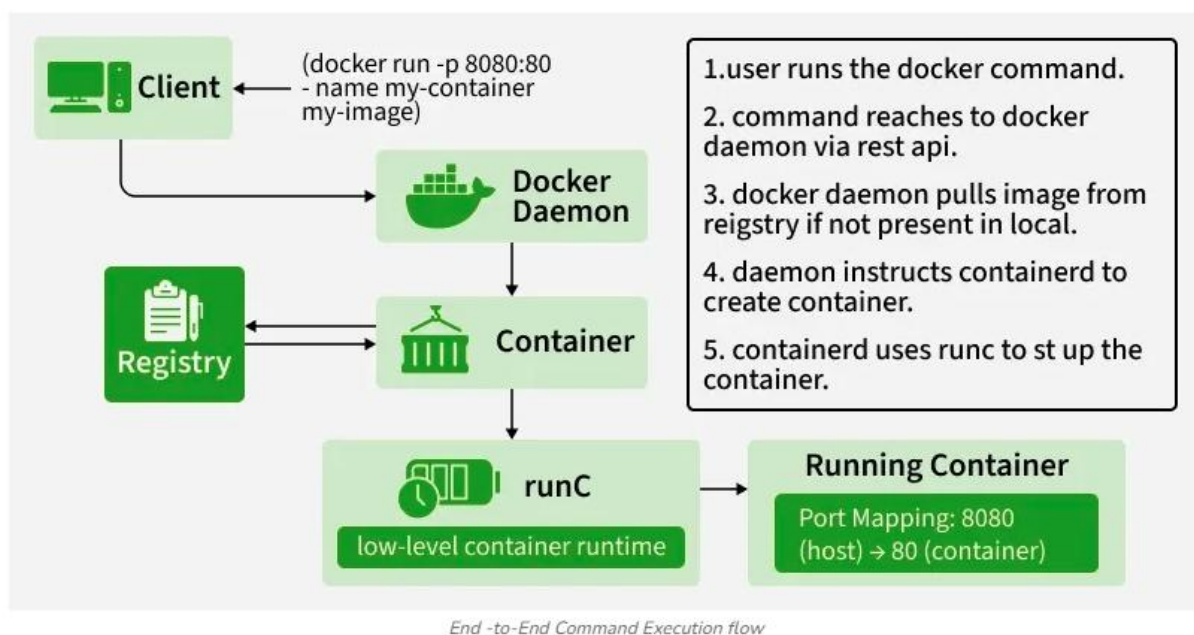
Types of Docker Network

1. **Bridge:** It is the default network driver. We can use this when different containers communicate with the same docker host.
2. **Host:** When you don't need any isolation between the container and host then it is used.
3. **Overlay:** For communication with each other, it will enable the swarm services.
4. **None:** It disables all networking.

5. **macvlan**: Assigns a unique MAC address to a container, making it appear as a physical device on your network.

Step-by-Step Execution of a Docker Command

Let's trace a common command to understand how all components work together:



You run the command: `docker run -d -p 80:80 nginx`

1. **Client**: The Docker Client sends a REST API request to the Docker Daemon to create and run a container from the nginx image.
2. **Daemon**: The Daemon receives the request. It first checks if the nginx image exists locally on the Host.
3. **Registry (Pull)**: If the image is not found locally, the Daemon contacts the configured Registry (Docker Hub by default) and pulls the nginx image.
4. **Runtime (containerd)**: The Daemon passes the image and run-configuration over to containerd.

5. **Runtime (runc):** containerd uses runc to create a new container. runc interfaces with the Linux kernel to create isolated namespaces and limit resources with cgroups.
6. **Execution:** The container is started. Docker maps port 80 of the host to port 80 of the nginx container, as requested by the -p 80:80 flag. The Nginx process runs as PID 1 inside the container's isolated PID namespace.

Docker's architecture is a sophisticated ecosystem of components that work together seamlessly to enable containerization. By understanding the intricacies of Docker architecture, developers can leverage its power to build, ship, and run applications more efficiently and reliably. Whether you're a seasoned Docker user or new to containerization, grasping the fundamentals of Docker architecture is essential for mastering modern software development and deployment practices.